FPGA Based FIR Filter with a UART Interface

EC551 Team Project Proposal

Boston University

BOSTON UNIVERSITY

Francis, Dill, Tanner, Alex :] Email: lirifx@bu.edu

2026 Fall

What does a FIR Filter do?

Nonrecursive filter that outputs a signal based on the previous inputs.

The output signal represents an averaged out or ‘smoothed’ signal that contains less noise.

Finite: Output settles to 0 in a finite amount of time when input is a single 1 followed by 0s.

BOSTON UNIVERSITY

Why Should We Care About This?

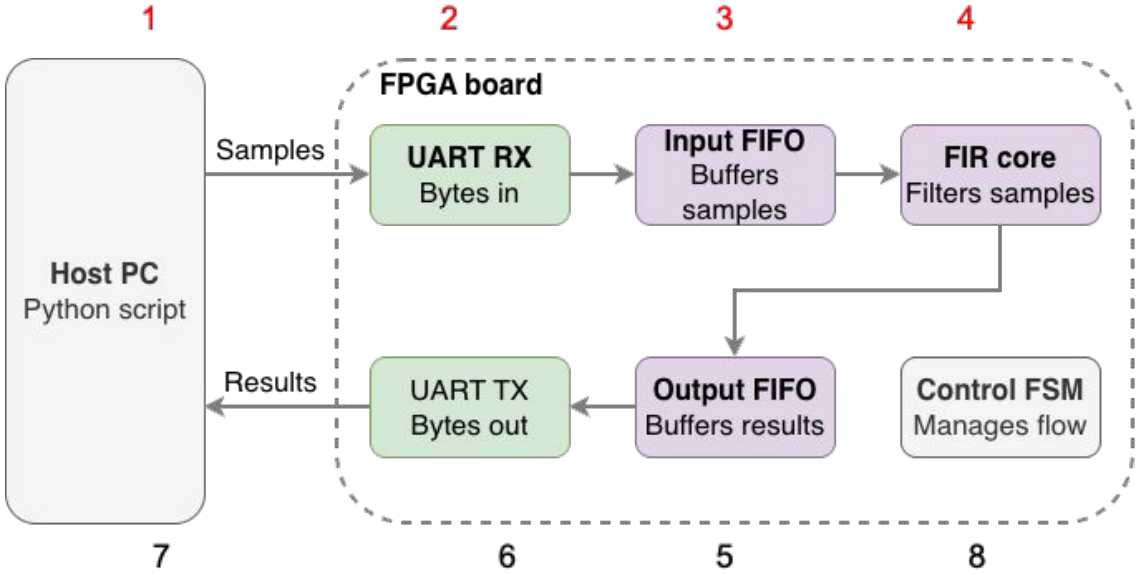
- FIR filters are used everywhere: audio processing, wireless systems, medical devices
- FIR filters are stable and can have linear phase, which is why they're preferred in many applications
- FPGAs are a natural fit. FIR filters use many multiply-accumulate operations which can be run in parallel and use DSP blocks. High throughput and predictable latency
- Industry relevant

Goal of the Project

- Design a parameterizable FIR filter
- Implement it on an FPGA to filter a real signal in real time
- Use fixed-point arithmetic and pipelining to reach high throughput
- Verify correctness and compare latency with a software reference
- Stretch goal: compare different architectures on speed, resource usage, and accuracy

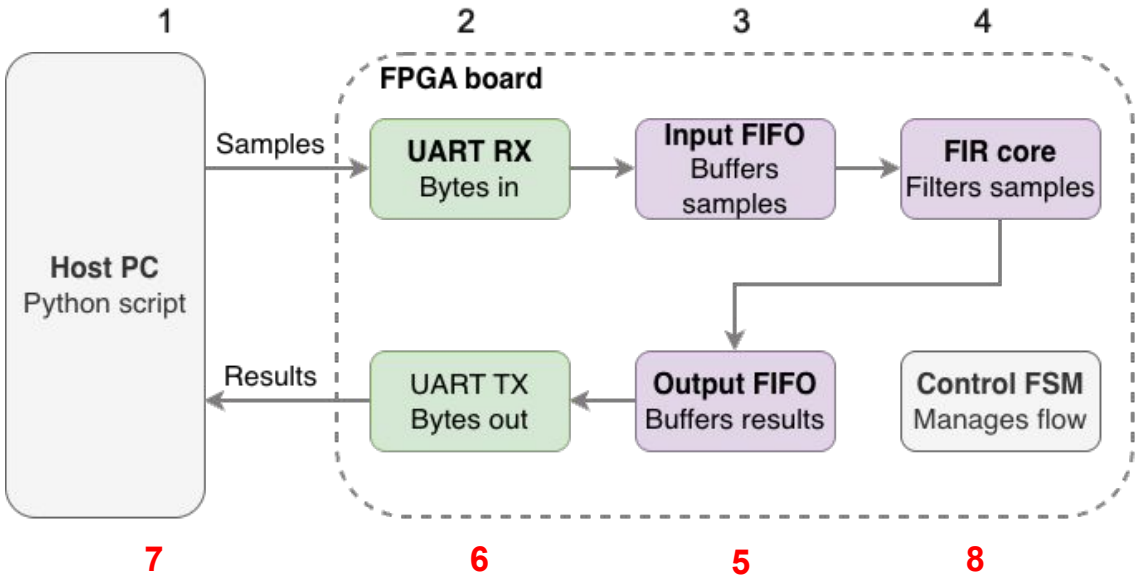
System Diagram

1. Host PC python script sends test sample bytes via the USB UART link
   - ❏ Universal Asynchronous Receiver-Transmitter (UART)
   - ❏ Example of test sample is noisy sine wave
2. UART receiver reassembles bytes into samples
3. Input FIFO holds bytes till filter in FIR core is ready
4. FIR core processes samples



System Diagram

5.Results wait in output FIFO since UART is slower than filter
6.UART transmitted sends results back out to host PC
7.Host PC plots results against python reference model and compares them
8.Control FSM coordinates operations
    Starting and pausing data flow based on FIFO's full/empty flags
    Combining 2 bytes into 1 16-bit sample

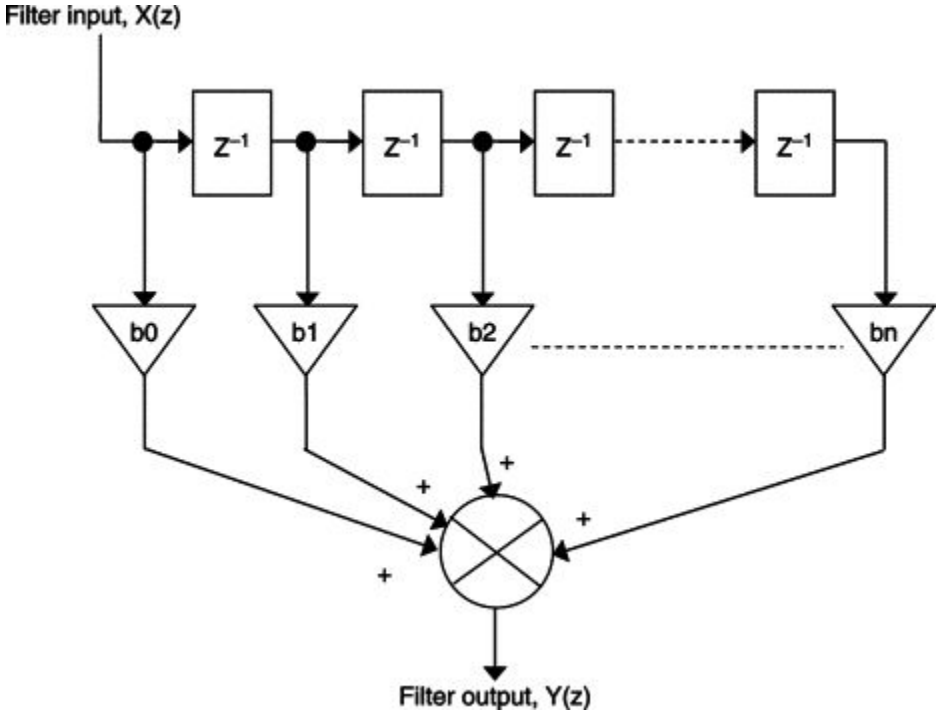


FIR: Finite Impulse Response UART: Universal Asynchronous Receiver/Transmitter

BOSTON UNIVERSITY

FIR core

Figure 7.43



BOSTON UNIVERSITY

System Demo

- Python simulation verification
  - Action: Show screenshot showing a UART frame received and testbench output
  - Expected result: Shows design was verified in simulation
- Live hardware demo
  - Step 1: Test impulse response
    - Action: Send single sample value of 1+0's (depending on the fixed point format) eg 100000
    - Expected result: FIR filters impulse response is exactly its coefficients so FPGA should send back coefficient list
  - Step 2: Noisy signal cleanup
    - Action: Send low-frequency sine wave with high frequency added noise and plot input and FPGA output on same graph
    - Expected result: Output should be a clean, slightly delayed, sine wave

FIR: Finite Impulse Response UART: Universal Asynchronous Receiver/Transmitter

System Demo

- Step 3: Bit-exact comparison
  - Action: Send same low-frequency sine wave + high frequency noise though the Python fixed-point mode and compare Python output to the FPGA output sample by sample
  - Expected result: Plot/print of mismatch count should be 0 proving correctness
- Step 4: Frequency response
  - Action: Send a series of sine waves at different frequencies, measure each output amplitude
  - Expected result: Plot of measured frequency response and python theoretical response should have overlapping curves
    - Shows the hardware behaves as designed across the whole spectrum
- Step 5: Stress Test
  - Action: Send a large random input (100,000 samples) and check for zero mismatches
  - Expected result: No dropped bytes, FIFO overflows or timing glitches

Resources Needed for the Project

FPGA development board

- Implements the filter's registers, multipliers, address, memory, and control logic

FPGA development software

- Synthesis, implementation, timing analysis, bitstream generation

Verilog simulator

- Test the filter before hardware implementation

Laptop or desktop

- Write Verilog, run simulations, program the FPGA, generate datasets, and plot results

Python or MATLAB

- Design coefficients, generate test signals, calculate expected outputs,and plot results

Git repository

- Share code and track changes across the team

Biggest Worries and Backup Plan

- FIR filter code doesn't work/not feasible to complete in time:
  - Try simplifying scope or logic if possible
  - Discuss with Prof.
  - Pivot to different project?
- Verilog code simulates but does not synthesize.
  - Comb through code for time-based code, behavioural verilog, loops with varying bounds, and other non-synthesizable code
  - Discuss with Prof.
- Output from module not compatible with Output Processing/graphing:
  - Simplify scope - copy/paste data into spreadsheet and attempt to display on graph there.
  - Simplify further - can effects be seen from the waveforms themselves?
- Input to module (pre-graphing, automatic entry) not compatible with module:
  - Disconnect graphing and display of unsmoothed data from data entry to actual module.
  - Attempt to make project work by passing in values manually.
- Assigning Work & Potential Team Dysfunction

Project Timeline and Assignments

Stage 1: Project Planning — September 29-October 2

- Define the project goal and high-level system diagram.
- Outline the demonstration and identify hardware/software needs.
- Assign team responsibilities and identify risks and backup plans
-

Stage 2: Filter Specifications — October 2–7

- Choose a low-pass, high-pass, or band-pass response
- Select sample rate, cutoff frequencies, and number of taps
- Define input/coefficient bit widths and target sample throughput

Stage 3: Software Reference Model — October 7–16

- Model the filter in Python or MATLAB.
- Generate filter coefficients and test signals.
- Convert samples and coefficients to fixed-point representations.
- Generate expected outputs for comparison with the Verilog implementation.

Stage 4: Verilog Implementation and Simulation — October 16–30

- Implement the sample delay line, coefficient storage, multipliers, and accumulator.
- Add control logic and reset behavior.
- Create a testbench.
- Compare simulation results with the software reference model.

Stage 5: FPGA Integration — October 30–November 18

- Implement sample input/output and board connections.
- Add clock and reset handling.
- Synthesize, implement, and check timing.
- Program the FPGA and capture filtered outputs.

Stage 6: Demo and Documentation — November 20–25

- Create before-and-after waveform and frequency plots from FPGA results.
- Document the architecture, performance, limitations, and operating procedure

References used

https://www.youtube.com/@advanceddigitaldesignwithv4311

https://www.dspguide.com/ch14/1.htm

https://e2e.ti.com/cfs-file/__key/communityserver-discussions-components-files/171/Lab6.pdf (P8-10, P25)

https://www.sciencedirect.com/topics/computer-science/finite-impulse-response-filter (Info and figure)

https://www.bohrium.com/en/sciencepedia/feynman/keyword/pulse_transfer_function